

Relazione Progetto di Programmazione ad Oggetti

Coffee Keys Logistics

Alberto Cavalletto - Matricola 2116418

Progetto svolto in coppia con Andrea Nogara - Matricola 2101078

Anno accademico 2025/2026

Indice

1	Introduzione	2
2	Classi principali del modello logico	2
3	Polimorfismo	2
4	Persistenza dei dati	3
5	Funzionalità aggiuntive	3
6	Ambiente di sviluppo ed esecuzione	3
7	Rendicontazione ore previste e svolte	3
8	Suddivisione delle attività tra i membri del gruppo	4

1 Introduzione

Coffee Keys è un negozio (esiste davvero) specializzato in tastiere meccaniche custom, switch e keycap set. L'idea è di Andrea, che segue già questo mondo delle tastiere custom da un po'. Un negozio online come questo non vende soltanto: c'è un magazzino da rifornire, ci sono i resi da rimandare in garanzia quando qualcosa arriva rotto, capita di mandare qualche tastiera in omaggio a un recensore per farsi conoscere, e poi ci sono i group buy, cioè quando più persone prenotano insieme un lotto di keycap che arriva tutto in blocco dopo qualche mese di attesa (una cosa piuttosto comune in quel settore). Il nostro progetto tiene traccia di tutte queste spedizioni. C'è una lista con tutte le spedizioni registrate, filtrabile in tempo reale scrivendo nella barra di ricerca; cliccandone una si apre il dettaglio con il manifesto specifico di quel tipo e, quando c'è, la foto del prodotto. Da lì si può modificarla, eliminarla, oppure tornare indietro e creare una spedizione nuova con un form che cambia i campi richiesti in base al tipo scelto dal menù a tendina.

Il progetto l'abbiamo fatto in coppia, seguendo l'architettura Model-View indicata dalle specifiche, senza Controller separato.

2 Classi principali del modello logico

Prima di metterci a scrivere codice ci siamo fatti la domanda: quali sono davvero i casi che un negozio come Coffee Keys deve gestire? Abbiamo cercato di far coincidere ogni classe concreta con uno di questi casi reali, invece di inventarne qualcuno a caso solo per arrivare al numero minimo di sottoclassi richiesto dalle specifiche.

La classe base è `Spedizione`, astratta, e contiene gli attributi in comune a tutte le spedizioni indipendentemente dal tipo: `id`, `destinatario`, `indirizzo`, `peso`, `prodotto` e `immagine`. Ha due metodi virtuali puri, `getManifestoSpedizione()` e `toJson()`, che ogni sottoclasse è costretta a implementare a modo suo. Da `Spedizione` derivano sei classi concrete.

`SpedizioneClientiStandard` è il caso più comune: un cliente ordina una tastiera e la riceve tramite un corriere qualsiasi, senza fretta particolare. Con `SpedizioneClientiEspressa` invece il cliente vuole la consegna garantita entro un certo numero di ore e paga qualcosa di più per quello. `SpedizioneReviewerSponsor` è un po' diversa dalle prime due perché non c'è un cliente che paga: il negozio manda in omaggio una tastiera a un recensore, di solito per farsi un po' di pubblicità online.

Le altre tre riguardano la merce che entra invece di uscire dal magazzino. `SpedizioneRifornimentoStock` è la merce che arriva da un fornitore per riempire lo stock. `SpedizioneResoFabbrica` sono invece i prodotti difettosi che bisogna rimandare al produttore per farsi rimborsare o sostituire in garanzia. Infine `SpedizioneGroupBuyInbound` rappresenta l'arrivo di un lotto da un group buy: nel mondo delle tastiere custom è comune che più persone prenotino insieme un lotto di produzione (per esempio un set di keycap) versando un acconto, e il lotto arrivi tutto insieme dopo qualche mese.

Ognuna delle sei ha poi i suoi attributi specifici, coerenti col caso che rappresenta: `oreConsegnaGarantite` per l'espressa, `codiceRMA` e `motivoReso` per il reso in fabbrica, `idLottoProduzione` e `accontoVersato` per il group buy, e così via per le altre.

Fuori dalla gerarchia c'è `HubLogistica`, che gestisce l'elenco delle spedizioni in memoria tramite un vector di puntatori a `Spedizione`: aggiunta, rimozione, ricerca per `id`, e libera tutta la memoria nel distruttore quando l'applicazione chiude.

3 Polimorfismo

`HubLogistica` deve tenere insieme sei tipi di spedizione diversi nello stesso vettore, e non volevamo scrivere un controllo diverso in base al tipo ogni volta che serviva fare qualcosa. Abbiamo quindi lasciato che fosse ogni sottoclasse a sapere come comportarsi da sola, tramite due metodi virtuali non banali:

- `getManifestoSpedizione()`: scrive il testo che si vede nel dettaglio, con campi diversi per ogni tipo (es. codice RMA e garanzia per un reso, data e lotto per un group buy)
- `toJson()`: salva gli attributi dell'oggetto in un `JsonObject`, scrivendo solo i campi che servono a quel tipo

In questo modo né la GUI né `HubLogistica` devono sapere che tipo di spedizione stanno gestendo in un dato momento: chiamano questi due metodi tramite il puntatore alla classe base, e viene fuori il comportamento giusto per ogni tipo senza bisogno di `if` o `switch` sul tipo dell'oggetto.

Il campo "tipo" che viene scritto da `toJson()` non sostituisce il polimorfismo: si usa solo quando si carica un file, in `JsonConverter` (o in `XmlPersistenza` per l'XML), per capire quale costruttore chiamare e ricreare l'oggetto giusto. Da un file non si può capire il tipo C++ originale in nessun altro modo. Una volta che l'oggetto esiste, tutto il resto passa di nuovo dai metodi virtuali.

4 Persistenza dei dati

Abbiamo fatto due formati di persistenza, come richiesto per i progetti in coppia. Il JSON è servito principalmente per salvare e ricaricare lo stato dell'applicazione così com'è (è comodo perché `QJsonObject` lo gestisce quasi da solo), mentre l'XML lo abbiamo pensato più come formato di scambio: è un po' più verboso ma anche più leggibile da un occhio esterno o da un altro programma che dovesse leggere gli stessi dati senza conoscere la nostra applicazione.

- JSON: `JsonFile` legge e scrive il file su disco, `JsonConverter` converte una `Spedizione` da/verso `QJsonObject`, `JsonReader` mette insieme i due per caricare o salvare tutta la lista
- XML: `XmlPersistenza` legge e scrive con `QXmlStreamReader/QXmlStreamWriter`, con una logica per ricostruire gli oggetti indipendente da quella del JSON

Il caricamento e il salvataggio si fanno sempre dal menu File (Apri, Salva, Salva Come), che apre una finestra di dialogo. Non ci sono percorsi di file fissi scritti nel codice: il formato si capisce dall'estensione scelta dall'utente.

5 Funzionalità aggiuntive

- Ricerca parziale, case-insensitive e in tempo reale su id, prodotto, destinatario e indirizzo
- Form di creazione/modifica con campi che cambiano dinamicamente in base al tipo di spedizione selezionato
- Gestione di un'immagine del prodotto: selezione da file, copia automatica nella cartella Resources del progetto e anteprima nella vista di dettaglio, con messaggio di fallback se l'immagine non è disponibile
- Richiesta di conferma prima di eliminare una spedizione o di perdere modifiche non salvate

6 Ambiente di sviluppo ed esecuzione

Abbiamo sviluppato il progetto su Windows, con Qt Creator, Qt 6.11.1 e il compilatore MinGW. Il progetto deve compilare anche nel container Docker del corso, quindi non ci siamo fidati solo del nostro PC: abbiamo installato la stessa immagine Ubuntu con lo stesso Qt e lo stesso compilatore indicati nel Dockerfile su una macchina virtuale, e per fortuna il progetto si è compilato senza errori né warning anche lì.

All'avvio l'applicazione parte senza spedizioni salvate: non abbiamo lasciato dati precaricati nel codice. Per questo abbiamo preparato, nella cartella Esempi fornita assieme al progetto, due file con una spedizione per ciascuno dei sei tipi: si possono caricare dal menu File per popolare subito l'applicazione e provare tutte le funzionalità senza dover inserire i dati a mano.

7 Rendicontazione ore previste e svolte

Tabella delle ore individuali (componente: Alberto Cavalletto):

Fase	Previste	Effettive	Note
Progettazione	7 h	7 h	Analisi requisiti dominio, progettazione gerarchia classi del Modello, cooperazione al diagramma UML. Concluso nei tempi previsti.
Sviluppo Modello	11 h	13 h	Scrittura gerarchia Spedizione e sotto-classi, gestione del polimorfismo. +2h per raffinare i metodi virtuali puri e l'uso corretto del polimorfismo.
Persistenza Dati	12 h	14 h	Moduli di salvataggio/caricamento JSON e XML. +2h per il debug del parsing XML e la gestione delle eccezioni.
Integrazione GUI	5 h	6 h	Supporto al collegamento tra Modello e View/segnali-slot. +1h per verificare la sincronizzazione dei dati tra vista e memoria.
Testing & Fix	4 h	5 h	Test di tenuta della logica, verifica dei flussi, rifinitura. +1h per garantire l'assenza di memory leak.
Relazione	3 h	3 h	Stesura e revisione della relazione individuale. Concluso nei tempi previsti.
Totale	42 h	48 h	

La parte che ci ha portato via più tempo del previsto è stata la persistenza dei dati. Il JSON, grazie a `QJsonObject` e `QJsonDocument`, si è scritto abbastanza in fretta, ma quando siamo passati all'XML ci siamo accorti che `QXmlStreamReader` va gestito con più attenzione: bisogna seguire l'apertura e la chiusura di ogni singolo tag a mano, e all'inizio abbiamo perso un po' di tempo con dei parsing che si fermavano a metà file senza un errore chiaro. Anche lo sviluppo del Modello ha richiesto qualche ora in più di quanto avevamo preventivato, soprattutto per essere sicuri che i due metodi virtuali puri bastassero davvero a coprire tutti i sei casi senza dover mettere neanche un controllo sul tipo da qualche parte per pigrizia. Le altre fasi invece sono andate più o meno secondo i tempi che ci eravamo dati all'inizio.

8 Suddivisione delle attività tra i membri del gruppo

Alberto - Modello logico e persistenza

- Progettazione e scrittura della classe base `Spedizione` e delle sei classi concrete derivate
- Implementazione di `HubLogistica`, il contenitore che gestisce in memoria l'elenco delle spedizioni (aggiunta, rimozione, ricerca)
- Sviluppo della persistenza JSON (`JsonFile`, `JsonConverter`, `JsonReader`)
- Sviluppo della persistenza XML (`XmlPersistenza`), secondo formato richiesto per i progetti in coppia

Andrea - Interfaccia grafica

- Sviluppo di `MainWindow`, con la creazione dei menu (File, Spedizione, Modifica) e la gestione della navigazione tra le schermate
- Realizzazione di `ListaSpedizioniWidget` (elenco e ricerca) e `DettaglioSpedizioneWidget` (visualizzazione del manifesto polimorfo)
- Realizzazione di `FormSpedizioneWidget`, con i campi che cambiano dinamicamente in base al tipo di spedizione selezionato
- Cura della disposizione dei layout e dell'aspetto dei pannelli

Attività svolte insieme

- Analisi del dominio logistico e progettazione della gerarchia delle classi, con relativo diagramma UML
- Collegamento tra le view e il Model all'interno di `MainWindow`, tramite il meccanismo di segnali e slot di Qt
- Testing dell'applicazione a runtime e verifica dell'assenza di memory leak